DEOS Core Protocol Architecture

Sürüm 1.1 – Medya Bağımsız Çekirdek Protokol

Emir Arkalı (emirarkali07@gmail.com)
DEOS – Dokuz Eylül Otonom Sistemler

İlk Yayın: 28 Eylül 2026 | Son Revizyon: 13 Ekim 2026

Doküman Tipi:	Core Protocol Specification & Architecture	**Doküman Kodu:**	`DOC-ICD-CORE-001`
Doküman Sürümü:	Sürüm 1.1	**Protokol Sürümü:**	v1.1 (`0x11`) – Medya Bağımsız
Yazar:	Emir Arkalı (`emirarkali07@gmail.com`)	**Son Değiştiren:**	Emir Arkalı
İlk Yayın Tarihi:	28 Eylül 2026	**Son Revizyon Tarihi:**	13 Ekim 2026
Durum:	Yürürlükte / Onaylandı	**Taşıma / Kapsam:**	Medya Bağımsız Çekirdek Mimari

Doküman Revizyon Geçmişi

Revizyon	Tarih	Yazar / Değiştiren	Açıklama
v0.3	28 Eylül 2026	Emir Arkalı	İlk taslak ve monolitik ICD v0.3 tanımı.
v1.0	04 Ekim 2026	Emir Arkalı	Modüler ICD ayrımı ve çekirdek mimari taslağı.
v1.1	13 Ekim 2026	Emir Arkalı	Medya bağımsız mimari (Universal Frame) kesinleştirildi, CAN-FD detayları ilgili adaptöre devredildi, State/Mode Registry eklendi.

1 Amaç ve Kapsam

Bu doküman, Dokuz Eylül Otonom Sistemler (DEOS) dağıtık araç sisteminde kullanılan **medya bağımsız çekirdek protokolü** tanımlar.

DEOS protokolü yalnızca belirli bir donanıma (CAN-FD vb.) bağlı bir hat protokolü değildir; araç kontrol bilgisayarları (STM32), telemetri birimleri (LoRa), gömülü Linux / Raspberry Pi sistemleri (Ethernet TCP/UDP) ve otonom sürüş yazılımları (ROS 2) arasında ortak çalışan üst düzey bir uygulama ve taşıma mimarisidir.

Bu dokümanda:

- Mantıksal Düğüm Adresleme (Node Address Registry),
- Evrensel Mesaj Öncelik Mimarisi (Priority Registry),
- Mesaj Davranış Sınıfları (Message Class Registry),
- Fonksiyonel Servisler (Service Registry ve Node–Service İlişkisi),
- Medya Bağımsız Evrensel Taşıma Çerçevesi (Universal DEOS Transport Frame),
- İstek–Cevap (Request–Response) ve Konfigürasyon semantiği

tanımlanmaktadır.

Fiziksel taşıma ortamlarına özel donanımsal kodlama ve çerçeveleme ayrıntıları (CAN-FD için 29-bit CAN ID ve DLC yuvarlama, LoRa için UART senkronizasyonu ve CRC, Ethernet için TCP akış çerçeveleme) ilgili **Taşıma Adaptörü (Transport Adapter)** dokümanlarında tanımlanmıştır.

Bir Node ID fiziksel bir mikrodenetleyiciye birebir bağlı olmak zorunda değildir. Aynı fiziksel cihaz üzerinde birden fazla mantıksal DEOS Node ID bulunabilir; her Node ID sistemde bağımsız bir mantıksal protokol kimliğidir.

2 Namespace Hiyerarşisi

DEOS protokolünde bir mesajın kimliği tek bir "mesaj numarası" ile değil, katmanlı bir namespace bileşimiyle belirlenir:

```
DEOS_PROTOCOL
|
+-- NODE
|   +-- SOURCE_NODE
|   +-- DESTINATION_NODE
|
+-- PRIORITY
|
+-- MESSAGE_CLASS
|
+-- SERVICE
|   +-- COMMAND
|       +-- PAYLOAD
|           +-- PARAMETER_ID   (gerektiğinde)
|           +-- VALUE          (gerektiğinde)
|
+-- PROTOCOL_METADATA
    +-- VERSION
    +-- SEQUENCE
    +-- PAYLOAD_LENGTH
```

Buna göre bir mesajın semantik kimliği genel olarak şu bileşenlerden oluşur:

`Priority + Message Class + Service + Destination + Source + Command`

Parametre yönetimi yapılan mesajlarda buna ek olarak `Parameter ID` kullanılır.

3 Mesaj Öncelik Mimarisi (Priority Registry)

DEOS çekirdeğinde (`deos_core`) ve tüm taşıma çerçevelerinde Öncelik (`Priority`) alanı **1 Byte (`uint8_t`)** olarak tanımlanır ve taşınır. Protokol standardı olarak bu 1 baytlık alan `0x00` ile `0x07` ($0 - 7$) arasındaki değerleri alır.

Öncelik değeri; gerek donanımsal bus arbitration (CAN-FD'ye aktarılırken 29-bit ID'nin 3 bitlik alanına haritalanır) gerekse yazılımsal kuyruk (Message Queue, LoRa TX, Router) seviyesinde mesajın öncelik sırasını belirler.

Sayısal olarak daha küçük olan değer her zaman daha yüksek önceliği temsil eder:

Değer	Symbol	Kullanım
0	`DEOS_PRIO_EMERGENCY`	Acil durdurma ve en kritik safety mesajları (E-Stop)
1	`DEOS_PRIO_CRITICAL_CONTROL`	Kritik gerçek zamanlı kontrol
2	`DEOS_PRIO_CONTROL`	Normal sürüş kontrol mesajları
3	`DEOS_PRIO_STATUS`	Durum ve geri besleme mesajları
4	`DEOS_PRIO_CONFIG`	Konfigürasyon ve kalibrasyon
5	`DEOS_PRIO_NETWORK`	Network management / heartbeat / ping-pong
6	`DEOS_PRIO_LOW`	Düşük öncelikli servis trafiği
7	`DEOS_PRIO_BACKGROUND`	Arka plan / geliştirme trafiği

4 Node Address Registry

Her fiziksel veya sanal DEOS haberleşme düğümüne sistem genelinde benzersiz bir 8-bit Logical Node ID atanır. Node ID, düğümün fiziksel donanım adresi veya transport adresi değildir; protokol içerisindeki mantıksal kimliğidir.

4.1 Adres Aralıkları

Adres Aralığı	Kullanım
0x00	Invalid / Unassigned
0x01–0x0F	Merkezi sistem ve virtual node'lar
0x10–0x1F	Aktüatör / motor ECU düğümleri
0x20–0x2F	Güç sistemi / BMS düğümleri
0x30–0x3F	Gateway / protocol bridge düğümleri
0x40–0x5F	Sensör düğümleri için ayrılmış alan
0x60–0xDF	Gelecek genişleme
0xE0–0xEF	Geliştirme / test düğümleri
0xF0–0xFD	Diagnostik ve servis araçları
0xFE	CAN Broadcast (Sadece CAN arayüzü)
0xFF	Global Broadcast (Tüm arayüzler)

4.2 Virtual Node Atamaları

Virtual node, bağımsız bir fiziksel mikrodenetleyici veya bilgisayar olmak zorunda olmayan, ancak DEOS protokolünde kendi Source ve Destination Node ID'si ile adreslenebilen mantıksal haberleşme ucudur.

Node ID	Symbol	Çalıştığı Ortam	Görev
0x01	DEOS_NODE_TEXTUAL	Raspberry Pi / Textual	Operatör arayüzü; TCP 5000 soketi üzerinden config, komut, status ve log sink
0x02	DEOS_NODE_GROUND_CONTROL	Uzak yer kontrol yazılımı	LoRa üzerinden telemetri ve yer istasyonu kontrol ucu
0x04	DEOS_NODE_ROS_BRIDGE	Gömülü Linux / ROS 2	Ethernet UDP 5001 portu üzerinden ROS 2 Köprüsü (deos_ros_bridge), otonom sürüş entegrasyonu

Textual Node yalnızca konfigürasyon amacıyla kullanılan bir node değildir. Protokolün izin verdiği servislerde komut gönderebilir, cevap alabilir, durum bilgilerini işleyebilir ve log mesajlarının doğrudan hedefi olabilir. Benzer şekilde ROS 2 Bridge ve Ground Control Node'ları da ihtiyaç duydukları servisleri kaynak veya hedef olarak kullanabilir.

4.3 Physical (Fiziksel) Node Atamaları

Node ID	Symbol	Fiziksel Cihaz	Görev
0x03	DEOS_NODE_MAIN_STM32	STM32H563ZI	Araç ana fiziksel kontrol ve ağ yönlendirici (Router) düğümü
0x10	DEOS_NODE_TRACTION	STM32 Traction ECU	Tahrik / invertör sistemi kontrolü
0x11	DEOS_NODE_STEERING	STM32 Steering ECU	Direksiyon açısı ve motoru kontrolü
0x12	DEOS_NODE_BRAKE	STM32 Brake ECU	Mekanik ve hidrolik frenleme sistemi kontrolü
0x30	DEOS_NODE_BMS_GATEWAY	CAN Gateway Donanımı	Classic CAN (Daly) ile CAN-FD omurgası arası donanımsal köprü

4.4 Hosted / Sanal (Virtual Local) Node Atamaları

DEOS protokolünde bir fiziksel mikrodenetleyici birden fazla mantıksal düğüme ev sahipliği yapabilir (deos_register_local_r

Node ID	Symbol	Barındıran Fiziksel Düğüm	Görev ve Açıklama
0x13	DEOS_NODE_IMU	DEOS_NODE_MAIN_STM32 (0x03)	Araç ana kartı üzerindeki 6-eksen ataletsel ölçüm sensörü telemetrisi
0x20	DEOS_NODE_BMS_MAIN	DEOS_NODE_BMS_GATEWAY (0x30)	Daly BMS #1 (Ana Yüksek Gerilim Çekiş Bataryası) sanal proxy düğümü
0x21	DEOS_NODE_BMS_AUX	DEOS_NODE_BMS_GATEWAY (0x30)	Daly BMS #2 (12V/24V Yardımcı Düşük Gerilim Bataryası) sanal proxy düğümü

4.5 Servis ve Özel Adres Atamaları

Node ID	Symbol	Tür	Görev
0xF0	DEOS_NODE_DIAG_TOOL	Servis aracı	Harici diagnostik ve test cihazı arayüzü
0xFE	DEOS_NODE_CAN_BROADCAST	Mantıksal Yayın	Sadece donanımsal CAN-FD hattındaki düğümlere yayın (Heartbeat vb.)
0xFF	DEOS_NODE_GLOBAL_BROADCAST	Mantıksal Yayın	CAN, LoRa, Ethernet dahil tüm ağ arayüzlerine genel yayın (E-STOP, State)

4.6 Node Enum Tanımı

```
typedef enum
{
    DEOS_NODE_INVALID         = 0x00,

    DEOS_NODE_TEXTUAL         = 0x01,
    DEOS_NODE_GROUND_CONTROL  = 0x02,
    DEOS_NODE_MAIN_STM32      = 0x03,
    DEOS_NODE_ROS_BRIDGE      = 0x04,

    DEOS_NODE_TRACTION        = 0x10,
    DEOS_NODE_STEERING        = 0x11,
    DEOS_NODE_BRAKE           = 0x12,
    DEOS_NODE_IMU             = 0x13,

    DEOS_NODE_BMS_MAIN        = 0x20,
    DEOS_NODE_BMS_AUX         = 0x21,

    DEOS_NODE_BMS_GATEWAY     = 0x30,

    DEOS_NODE_DIAG_TOOL       = 0xF0,

    DEOS_NODE_CAN_BROADCAST       = 0xFE,
    DEOS_NODE_GLOBAL_BROADCAST    = 0xFF,
    DEOS_NODE_BROADCAST           = 0xFF /* Alias */
} deos_node_id_t;
```

5 Message Class Registry

Message Class alanı mesajın hangi amaçla gönderildiğini tanımlar. Message Class, fonksiyonel servisten bağımsızdır.

ID	Symbol	Açıklama
0x0	DEOS_CLASS_SAFETY	Acil / safety-critical mesaj
0x1	DEOS_CLASS_COMMAND	Normal gerçek zamanlı kontrol komutu
0x2	DEOS_CLASS_STATUS	Durum / geri besleme / telemetri
0x3	DEOS_CLASS_EVENT	Fault veya olay bildirimi
0x4	DEOS_CLASS_REQUEST	Cevap beklenen okuma, sorgulama veya işlem isteği
0x5	DEOS_CLASS_RESPONSE	Bir REQUEST veya cevap gerektiren işleme verilen protokol cevabı
0x6	DEOS_CLASS_CONFIG	Parametre ve konfigürasyon işlemleri
0x7	DEOS_CLASS_NETWORK	Ağ yönetimi / heartbeat
0x8–0xF	Reserved	Gelecek kullanım

```
typedef enum
{
    DEOS_CLASS_SAFETY   = 0x0,
    DEOS_CLASS_COMMAND  = 0x1,
    DEOS_CLASS_STATUS   = 0x2,
    DEOS_CLASS_EVENT    = 0x3,
    DEOS_CLASS_REQUEST  = 0x4,
    DEOS_CLASS_RESPONSE = 0x5,
    DEOS_CLASS_CONFIG   = 0x6,
    DEOS_CLASS_NETWORK  = 0x7
} deos_message_class_t;
```

5.1 Message Class ve Service Arasındaki Fark

Message Class ile Service farklı sorulara cevap verir:

Alan	Cevapladığı soru	Örnek
Message Class	Mesaj node içerisinde hangi tür işlem akışına aittir?	COMMAND, CONFIG, NETWORK, RESPONSE
Service	Mesaj hangi fonksiyonel alana aittir?	STEERING, BRAKE, BMS, SYSTEM
Command	Bu fonksiyonel alan içerisinde tam olarak hangi işlem yapılacaktır?	SET_TARGET_ANGLE, SET_PARAMETER, PING

Aynı Service farklı Message Class'lar altında kullanılabilir. Örneğin STEERING servisi için:

```
COMMAND / STEERING / SET_TARGET_ANGLE
CONFIG  / STEERING / SET_PARAMETER
REQUEST / STEERING / GET_STATUS
```

Burada Service her üç mesajda da direksiyon fonksiyonel alanını belirtirken Message Class, mesajların node içerisinde farklı işlem yollarına ayrılmasını sağlar.

Tersi de mümkündür: aynı Message Class farklı Service'lerde kullanılabilir:

```
COMMAND / TRACTION / SET_TARGET_SPEED
COMMAND / STEERING / SET_TARGET_ANGLE
COMMAND / BRAKE    / SET_TARGET
```

Bu örneklerde Message Class tüm mesajların normal kontrol komutu olduğunu, Service ise hangi alt sisteme ait olduklarını belirtir.

6 Service Registry

Service alanı mesajın hangi fonksiyonel domaine ait olduğunu gösterir.

Service ID	Symbol	Amaç
0x00	DEOS_SERVICE_SYSTEM	Genel node state/mode/reset bilgileri
0x01	DEOS_SERVICE_TRACTION	Tahrik sistemi
0x02	DEOS_SERVICE_STEERING	Direksiyon sistemi
0x03	DEOS_SERVICE_BRAKE	Fren sistemi
0x04	DEOS_SERVICE_BMS	Batarya ve BMS verileri
0x05	DEOS_SERVICE_CALIBRATION	Kalibrasyon işlemleri
0x06	DEOS_SERVICE_DIAGNOSTIC	Fault / sağlık / diagnostik
0x07	DEOS_SERVICE_SENSORS	Sensör verileri
0x08	DEOS_SERVICE_LIGHTING	Aydınlatma ve sinyalizasyon sistemi
0x09–0x3F	Reserved	Yeni servisler için ayrılmış alan

```
typedef enum
{
    DEOS_SERVICE_SYSTEM      = 0x00,
    DEOS_SERVICE_TRACTION    = 0x01,
    DEOS_SERVICE_STEERING    = 0x02,
    DEOS_SERVICE_BRAKE       = 0x03,
    DEOS_SERVICE_BMS         = 0x04,
    DEOS_SERVICE_CALIBRATION = 0x05,
    DEOS_SERVICE_DIAGNOSTIC  = 0x06,
    DEOS_SERVICE_SENSORS     = 0x07,
    DEOS_SERVICE_LIGHTING    = 0x08
} deos_service_id_t;
```

7 Node–Service İlişkisi

Node ve Service aynı kavram değildir. Node, fiziksel veya sanal haberleşme düğümünü; Service ise mesajın ait olduğu fonksiyonel alanı tanımlar. Bir node'un belirli bir servise mesaj gönderebilmesi, o servisin kendisi üzerinde barındırıldığı anlamına gelmez.

Aşağıdaki tablo temel service handler sahipliğini gösterir. Her adreslenebilir DEOS node'u, ayrıca bu ICD'de tanımlanan ortak SYSTEM/NETWORK PING–PONG davranışını desteklemek zorundadır.

Node	SYS	TRAC	STEER	BRAKE	BMS	CAL	DIAG	LIGHT
TEXTUAL	X	–	–	–	–	–	X	–
GROUND_CONTROL	X	–	–	–	–	–	X	–
MAIN_STM32	X	X	X	X	X	X	X	X
MICRO_ROS	X	–	–	–	–	–	X	–
TRACTION	X	X	–	–	–	X	X	–
STEERING	X	–	X	–	–	X	X	–
BRAKE	X	–	–	X	–	X	X	–
BMS_MAIN	X	–	–	–	X	–	X	–
BMS_AUX	X	–	–	–	X	–	X	–
BMS_GATEWAY	X	–	–	–	X	–	X	–

Tablodaki işaretler, ilgili node'un o servise ait mesajları desteklediğini ifade eder; mesaj gönderme yeteneğini sınırlamaz. Örneğin `DEOS_NODE_TEXTUAL`, hedefi `DEOS_NODE_STEERING` olan ve `DEOS_SERVICE_STEERING` kullanan bir CONFIG veya REQUEST mesajı gönderebilir.

Örneğin direksiyon düğümüne normal sürüş komutu gönderilirken `Destination = DEOS_NODE_STEERING`, `Service = DEOS_SERVICE_STEERING` kullanılır. Aynı fiziksel düğüme kalibrasyon komutu gönderilirken `Destination = DEOS_NODE_STEERING`, `Service = DEOS_SERVICE_CALIBRATION` kullanılır.

8 Çoklu Taşıma Katmanı Mimarisi (Multi-Transport Architecture)

DEOS protokolü, alt taşıma katmanından tamamen soyutlanmış (transport-agnostic) olarak çalışır. Protokol çekirdeği (Core Dispatcher / Router), mesajları medya bağımsız mantıksal formatta (`deos_message_t` / `Universal DEOS Transport Frame`) işler ve hedef düğümün adresine göre ilgili fiziksel taşıma adaptörüne yönlendirir.

Sistemde tanımlı fiziksel taşıma adaptörleri ve ilgili ICD dokümanları şunlardır:

1. **CAN-FD Taşıma Adaptörü (`ICD_CANFD_Transport`):** Araç içi omurga haberleşmesidir. Çekirdek protokol alanları (Priority, Class, Service, Destination, Source) 29-bit Extended CAN Identifier alanına kodlanır. BRS (Bit Rate Switch) ve DLC yuvarlama kuralları bu adaptör tarafından yürütülür.
2. **Ethernet TCP/UDP Adaptörü (`ICD_Ethernet_TCP`):** Raspberry Pi, telemetri bilgisayarı ve teşhis araçları ile yüksek bant genişlikli IP iletişimidir. Akış ayrıştırması (stream parsing) için 1 baytlık çerçeve uzunluğu (`Length`) öneki kullanılır.
3. **LoRa UART Telemetri Adaptörü (`ICD_LoRa_Telemetry`):** Yer Kontrol İstasyonu ile araç arasındaki uzun mesafe kablosuz haberleşmedir. Gürültülü açık kanal nedeniyle senkronizasyon başlığı (`0xAA 0x55`), paket bayrakları (`Flag`) ve `CRC16-CCITT` koruma zarfı içerir.
4. **ROS 2 UDP Köprüsü:** Otonom sürüş yazılım katmanı (Navigasyon, Algılama) ile araç kontrol düğümleri arasında çift yönlü durum ve hedef aktarımı sağlar.

9 Medya Bağımsız Standart DEOS Taşıma Çerçevesi (Universal Transport Frame)

DEOS protokolü, CAN-FD donanımının 29-bit Arbitration ID ve bit paketleme kısıtlarından bağımsız olarak; Ethernet (TCP/UDP), LoRa (UART), ROS 2 köprüleri ve Yer Kontrol İstasyonu (Python) gibi tüm harici sistemlerle uçtan uca haberleşebilmek için standart bir **Evrensel Taşıma Çerçevesi (Universal DEOS Transport Frame)** tanımlar.

Dış ortamdan üretilen bir mesaj, CAN ID hesaplamasına veya donanımsal bit kaydırmalarına gerek kalmaksızın doğrudan bu standart dizilim ile paketlenir. Mesaj STM32 Ana Beyin (Router) katmanına ulaştığında, hedef CAN düğümüyse otomatik olarak 29-bit CAN ID ve CAN-FD verisine dönüştürülür.

9.1 Standart Çerçeve Anatomisi (Byte Byte)

Evrensel çerçeve, **9 Byte Sabit Başlık (Header)** ve ardından gelen N Byte **Payload** alanından oluşur:

Byte	Alan Adı	Tip	Boyut	Açıklama
0	`Destination Node`	`uint8_t`	1 Byte	Hedef düğüm kimliği (Örn: `0x03` Main STM32, `0x10` Traction, `0xFF` Broadcast).
1	`Source Node`	`uint8_t`	1 Byte	Kaynak düğüm kimliği (Örn: `0x02` Ground Control, `0x06` Textual UI).
2	`Priority`	`uint8_t`	1 Byte	Mesaj önceliği (`0x00` CRITICAL .. `0x05` NETWORK).
3	`Version`	`uint8_t`	1 Byte	DEOS protokol sürümü (Varsayılan: `0x11` = v1.1).
4	`Message Class`	`uint8_t`	1 Byte	Mesaj sınıfı (`0x01` COMMAND, `0x02` RESPONSE, `0x07` NETWORK vb.).
5	`Service ID`	`uint8_t`	1 Byte	Fonksiyonel servis (`0x00` SYSTEM, `0x01` TRACTION, `0x08` LIGHTING vb.).
6	`Command ID`	`uint8_t`	1 Byte	Servise ait komut (`0x01` TURN_ON, `0x02` TURN_OFF, `0x08` PING vb.).
7	`Sequence`	`uint8_t`	1 Byte	Mesaj takip sayacı (`0x00` – `0xFF`).
8	`Payload Length`	`uint8_t`	1 Byte	Arkadan gelen dinamik veri bayt sayısı (N, 0–60 byte).
9..	**Payload**	`uint8_t[`N`]`	N Byte	Gerçek komut/telemetri verisi (Örn: Far ID, Ping ID, Hız hedefi).

Toplam Paket Boyutu: $9 + N$ Byte.

9.2 Taşıma Katmanı Adaptasyonu

Standart Evrensel Çerçeve (9+N Byte), kullanılacak fiziksel taşıma hattına göre ilgili adaptör sürücüsü tarafından uygun şekilde paketlenir veya dönüştürülür:

- **CAN-FD Omurgası (`ICD_CANFD_Transport`):** Çerçevenin yönlendirme ve öncelik alanları (Destination, Source, Priority, Message Class, Service) 29-bit Extended CAN Identifier'a dönüştürülür. Çerçevenin geri kalan Version, Sequence, Command, Length ve Payload alanları CAN-FD veri alanına (Data Payload) yazılır. Donanımsal DLC yuvarlama ve BRS kuralları uygulanır.
- **TCP/Ethernet Arayüzü (`ICD_Ethernet_TCP`):** Akış tabanlı TCP soketinde çerçevenin başına 1 baytlık uzunluk öneki (`Length`) eklenerek `[Length (1B)] + [Standart DEOS Frame (9+N Byte)]` formatında iletilir.
- **LoRa / Seri Port Arayüzü (`ICD_LoRa_Telemetry`):** Senkronizasyon (`0xAA 0x55`), bayrak ve CRC16 kontrolü içeren paket yapısı ile sarmalanır.

9.3 Fiziksel İletim Katmanlarına Göre Dış Zarf (L2 Framing)

Evrensel 9+N baytlık standart DEOS çerçevesi, fiziksel iletim hattının özelliklerine göre dış zarflarla taşınır:

- **Ethernet (TCP):** TCP zaten akış kontrolü ve sağlama toplamı sağladığından, çerçevenin başına sadece toplam paket boyutu eklenir: `[Length (1B)] + [Standart DEOS Frame (9+N Byte)]`.
- **LoRa / Seri Port (UART):** Gürültülü ve bayt kayması olabilecek hatlarda senkronizasyon ve CRC doğrulaması eklenir: `[0xAA 0x55] [Flag] [Length] + [Standart DEOS Frame (9+N Byte)] + [CRC16 (2B)]`.
- **ROS 2 / UDP:** Doğrudan `[Standart DEOS Frame (9+N Byte)]` datagram olarak iletilir.

10 REQUEST–RESPONSE ve CONFIG Ayrımı

REQUEST, RESPONSE ve CONFIG birbirinden bağımsız mesaj sınıflarıdır ve farklı amaçlara hizmet eder.

- `REQUEST`: Bir düğümden veri, durum veya belirli bir işlem sonucu talep etmek için kullanılır.
- `RESPONSE`: REQUEST sınıfındaki bir mesaja veya cevap gerektiren başka bir işleme karşılık dönen cevabı taşır.
- `CONFIG`: Bir düğümün çalışma parametrelerini, ayarlarını veya kalıcı yapılandırmasını değiştirmeye yönelik mesajları taşır.

Örneğin bir parametreyi okumak için REQUEST, aynı parametreyi değiştirmek için CONFIG kullanılabilir:

```
REQUEST / STEERING / GET_PARAMETER / PID_KP
CONFIG  / STEERING / SET_PARAMETER / PID_KP
```

Her iki işlem için de gerektiğinde RESPONSE sınıfı ile sonuç döndürülebilir. Bu ayrım, mesajın "cevap bekleyip beklemediği" ile "konfigürasyon amacı taşıyıp taşımadığı" bilgilerinin birbirine karıştırılmasını önler.

11 Response Altyapısı

RESPONSE Message Class protokolde ayrılmıştır; ancak hangi komutların zorunlu response üreteceği bu registry dokümanında kesinleştirilmemiştir.

Özellikle aşağıdaki işlemler gelecekte response kullanılabilecek adaylardır:

- Parameter GET/SET,
- state veya mode değişimi,
- kalibrasyon başlatma / kaydetme,

- fault reset / clear,
- node reset,
- servis tipi tek-seferlik yönetim komutları.

11.1 Generic Response Payload

`DEOS_CLASS_RESPONSE` mesajları varsayılan olarak standart bir veri yerleşimi kullanır. Eğer ilgili komut özel bir ek veri döndürmüyorsa, yanıt mesajının içeriği her zaman şu formatta olur:

Byte	Alan	Tip	Açıklama
0	Result Code	`uint8_t`	İşlem sonucu (`deos_result_t`)
1..N	Ek Veri (Opsiyonel)	Değişken	Komuta özgü veri

Result code `0x00` (SUCCESS) olduğunda işlem başarılı sayılır. Bu genel yapı sayesinde, her komut için özel bir response formatı tasarlamaya gerek kalmadan hata yönetimi standartlaştırılmıştır.

Aşağıda standart response result kodları listelenmiştir:

Kod	Anlam
`0x00`	SUCCESS
`0x01`	UNSUPPORTED
`0x02`	INVALID_PARAMETER
`0x03`	OUT_OF_RANGE
`0x04`	NOT_ALLOWED
`0x05`	BUSY
`0x06`	NOT_READY
`0x07`	INTERNAL_ERROR

Bu alanın kesin payload yerleşimi sonraki ICD revizyonunda mesaj bazında ayrıntılandırılacaktır.

12 State ve Mode Registry

DEOS protokolünde sistem durumları iki bağımsız kavramla ele alınır:

- **State (Durum):** Düğümün veya aracın anlık operasyonel hazırlık seviyesini belirtir (`INIT`, `STANDBY`, `READY`, `ACTIVE`, `FAULT` vb.).
- **Mode (Mod):** Düğümün veya aracın hangi kontrol davranışı ve otonomi mantığı altında çalıştığını belirtir (`NORMAL`, `MANUAL`, `AUTONOMOUS`, `SERVICE`, `TEST` vb.).

12.1 State Enum

Değer	Symbol
`0x00`	`DEOS_STATE_INIT`
`0x01`	`DEOS_STATE_STANDBY`
`0x02`	`DEOS_STATE_READY`
`0x03`	`DEOS_STATE_ACTIVE`
`0x04`	`DEOS_STATE_CALIBRATING`
`0x05`	`DEOS_STATE_SAFE`
`0x06`	`DEOS_STATE_FAULT`
`0xFF`	`DEOS_STATE_UNKNOWN`

```
typedef enum
{
    DEOS_STATE_INIT         = 0x00,
    DEOS_STATE_STANDBY      = 0x01,
    DEOS_STATE_READY        = 0x02,
    DEOS_STATE_ACTIVE       = 0x03,
    DEOS_STATE_CALIBRATING  = 0x04,
    DEOS_STATE_SAFE         = 0x05,
    DEOS_STATE_FAULT        = 0x06,
    DEOS_STATE_UNKNOWN      = 0xFF
} deos_state_t;
```

12.2 Mode Enum

Değer	Symbol
0x00	DEOS_MODE_NORMAL
0x01	DEOS_MODE_MANUAL
0x02	DEOS_MODE_AUTONOMOUS
0x03	DEOS_MODE_SERVICE
0x04	DEOS_MODE_TEST
0xFF	DEOS_MODE_UNKNOWN

```
typedef enum
{
    DEOS_MODE_NORMAL      = 0x00,
    DEOS_MODE_MANUAL      = 0x01,
    DEOS_MODE_AUTONOMOUS  = 0x02,
    DEOS_MODE_SERVICE     = 0x03,
    DEOS_MODE_TEST        = 0x04,
    DEOS_MODE_UNKNOWN     = 0xFF
} deos_mode_t;
```

13 Mantıksal Node Kimliği Kuralı

DEOS ağında her Source ve Destination Node ID bağımsız bir mantıksal protokol kimliğidir. Aynı fiziksel cihaz üzerinde birden fazla mantıksal Node ID bulunabilse bile, bir mesaj hangi Node ID adına üretiliyorsa çerçevenin Source alanında o Node ID kullanılmalıdır.

Bir node başka bir node'un kimliğiyle cevap üretmemelidir. Örneğin `Destination = DEOS_NODE_BMS_MAIN` olan geçerli bir PING'e verilen PONG mesajında `Source = DEOS_NODE_BMS_MAIN` kullanılmalıdır.

Bu kural fiziksel yerleşimden ve taşıma katmanından bağımsızdır; ağdaki tüm katılımcılar mantıksal Node ID'leri bağımsız haberleşme uçları olarak değerlendirir.

14 İsimlendirme Standardı

Tüm protokol sabitlerinde ortak prefix yapısı kullanılır:

```
DEOS_NODE_...
DEOS_PRIO_...
DEOS_CLASS_...
DEOS_SERVICE_...
DEOS_CMD_<SERVICE>_...
DEOS_PARAM_<SERVICE>_...
DEOS_STATE_...
DEOS_MODE_...
```

Örnekler:

```
DEOS_NODE_TEXTUAL
DEOS_NODE_GROUND_CONTROL
DEOS_NODE_MAIN_STM32
DEOS_NODE_ROS_BRIDGE
DEOS_NODE_STEERING
DEOS_CLASS_CONFIG
DEOS_SERVICE_STEERING
DEOS_CMD_STEERING_SET_TARGET_ANGLE
DEOS_CMD_SET_PARAMETER
DEOS_PARAM_STEERING_PID_KP
DEOS_STATE_ACTIVE
DEOS_MODE_AUTONOMOUS
```

15 Özet Registry

Mevcut DEOS Çekirdek Protokol kararları aşağıdaki şekilde özetlenebilir:

- DEOS protokolü medya bağımsızdır (CAN-FD, TCP/Ethernet, LoRa UART, ROS 2).
- Standart Evrensel Taşıma Çerçevesi (Universal Frame) 9 bayt sabit başlık ve N bayt dinamik payloaddan oluşur.
- 9-bayt sabit başlık sırasıyla: Destination Node, Source Node, Priority, Version, Message Class, Service ID, Command ID, Sequence ve Payload Length alanlarını içerir.
- DEOS Protocol v1.1 wire sürüm değeri `0x11`'dir.
- Node ID, sistem genelinde kullanılan 8-bit mantıksal protokol kimliğidir (`0x00`–`0xFF`).
- Bir fiziksel donanım üzerinde birden fazla mantıksal Node ID barınabilir; ağda her biri bağımsız bir uçtur.
- Priority alanı 0 (`0x00` CRITICAL) ile 5 (`0x05` NETWORK) arasında değer alır.
- Çoklu taşıma ortamlarına aktarım, ilgili Taşıma Adaptörleri (`ICD_CANFD_Transport`, `ICD_Ethernet_TCP`, `ICD_LoRa_Telemetry`) tarafından şeffaf olarak yürütülür.
- Fiziksel CAN-FD omurgasında Destination, Source, Priority, Class ve Service alanları 29-bit CAN Identifier'a kodlanır.
- Durum yönetimi birbirinden bağımsız State ve Mode enumarasyonları ile sağlanır.
- Varsayılan çok baytlı sayısal veri sırası Little-Endian'dır.
- Bir cevabın (RESPONSE) Source Node değeri, cevabı üreten mantıksal node kimliği olmalıdır.